KULLIYYAH OF ENGINEERING (KOE)

MECHATRONICS SYSTEM INTEGRATION (MCTA3203)

SEMESTER 1, 25/26

SECTION 1

PROJECT REPORT WEEK 4

TITLE:

Serial interfacing with microcontroller: Sensors and actuators

PREPARED BY :

NO	NAME	MATRIC NO
1	MUHAMMAD NAUFAL HAKIMI BIN IRWAN AFFANDI	2313041
2	MUHAMAD IQBAL BIN MD ISA	2313247
3	ADAM NABIL HANIFF BIN RUZAIMI	2313203

DATE OF SUBMISSION: _/_/2025

ABSTRACT

This experiment focuses on integrating multiple input and output devices with an Arduino Uno through serial communication. The system combines an MPU6050 motion sensor, an RFID module (MFRC522), a servo motor, and LED indicators to form a motion-activated access system. The experiment is divided into three main tasks: real-time motion tracking using the MPU6050, RFID-based user authentication, and full system integration that combines motion and RFID verification to control a servo motor. Python is used to visualize motion data and communicate with Arduino for access validation. Results show successful detection of circular motion and RFID authentication, enabling servo unlocking with LED feedback. This experiment demonstrates the effectiveness of serial communication in real-time sensor-actuator integration for smart control systems.

TABLE OF CONTENTS

Part 1: Motion Detection with MPU6050 (Task 1: Real-Time Motion Tracking Using MPU6050 Sensor)

1.0 INTRODUCTION

The MPU6050 is a 6-axis motion tracking sensor that integrates a 3-axis gyroscope and a 3-axis accelerometer. It is widely used in robotics, gesture recognition, and motion-controlled systems due to its precision and compatibility with microcontrollers.

In this experiment, the MPU6050 sensor is interfaced with the Arduino Uno through the I2C communication protocol. The Arduino collects real-time motion data and sends it via serial communication to Python, where the data is plotted using the Matplotlib library. The main goal is to detect a specific motion pattern — a **circular gesture** — which will be used later as a verification step in the integrated smart access system (Part 2).

2.0 MATERIALS AND EQUIPMENT

No	Component
1	Arduino Uno
2	MPU6050 Sensor
3	Jumper Wires
4	Breadboard
5	Computer with Arduino IDE & Python
6	USB Cable

3.0 EXPERIMENTAL SETUP

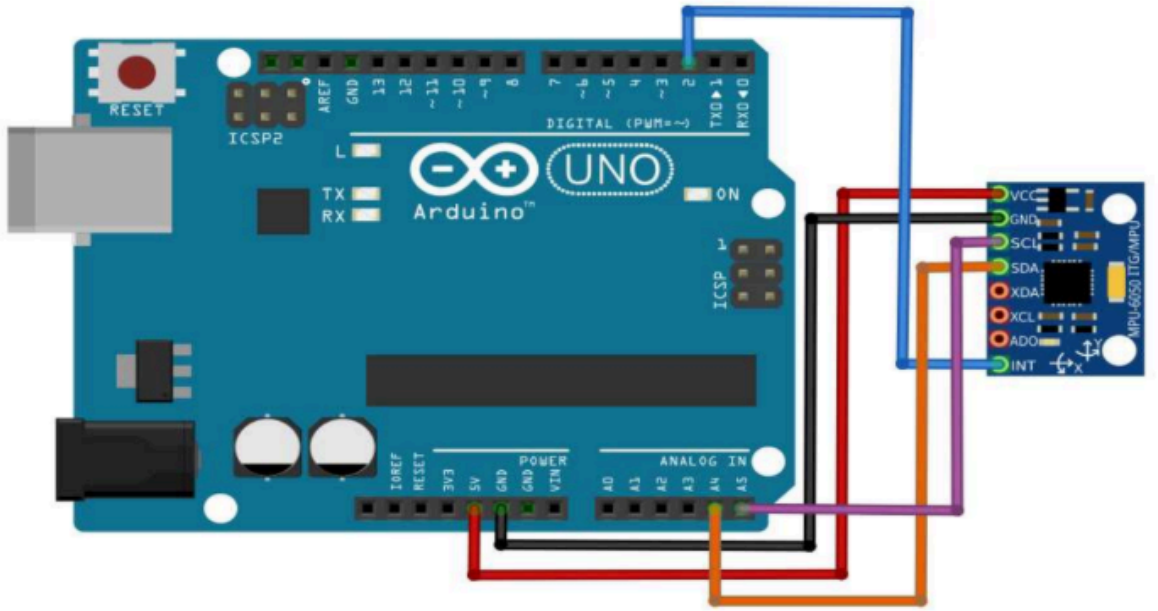


Fig 1: Arduino-MPU6050 connection

MPU6050 Pin	Arduino Uno Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

4.0 METHODOLOGY

The experiment integrates an MPU6050 sensor, Arduino Uno, and a Python program to perform real-time motion tracking. The MPU6050 was connected to the Arduino via I2C communication (SDA to A4, SCL to A5, 5V and GND for power).

The Arduino code initializes the MPU6050 using the MPU6050 library and continuously reads six motion parameters — three from the accelerometer (ax, ay, az) and three from the gyroscope (gx, gy, gz). These values are transmitted to the computer through the serial port in comma-separated format at a baud rate of 9600.

On the computer, the Python script establishes a serial connection with the Arduino and uses Matplotlib to visualize the motion in real time. The program reads incoming serial data, extracts the X and Y acceleration values, and dynamically plots them on a live graph.

Both systems work together through **serial communication**, where Arduino acts as the data sender and Python as the visualizer, enabling continuous monitoring of the MPU6050 sensor's motion path.

Coding (Python):

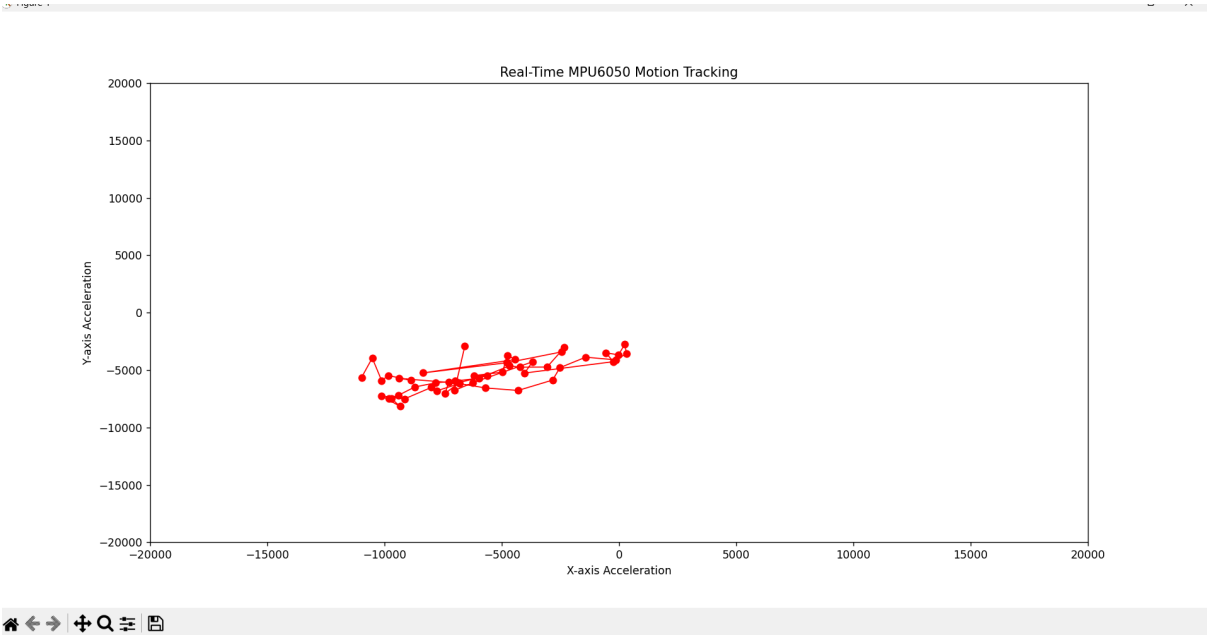
```python
import serial
import time
import matplotlib
matplotlib.use('TkAgg')   # ensures GUI plotting works on Windows
import matplotlib.pyplot as plt

# --- Serial setup ---
ser = serial.Serial('COM5', 9600, timeout=1)   # change COM port if needed
time.sleep(2)
print("✅ Connected to", ser.port)

# --- Matplotlib setup ---
plt.ion()
fig, ax = plt.subplots()
ax.set_title("Real-Time MPU6050 Motion Tracking")
ax.set_xlabel("X-axis Acceleration")
ax.set_ylabel("Y-axis Acceleration")
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)

(line,) = ax.plot([], [], "ro-", linewidth=1)
x_data, y_data = [], []

try:
    while True:
        data = ser.readline().decode(errors="ignore").strip()
        if not data:
            continue

        parts = data.split(",")
        if len(parts) >= 2:
            try:
                ax_val = int(parts[0])
                ay_val = int(parts[1])
            except ValueError:
                continue

```

```
                x_data.append(ax_val)
                y_data.append(ay_val)

                # keep only recent points
                x_data = x_data[-50:]
                y_data = y_data[-50:]

                line.set_data(x_data, y_data)
                ax.relim()
                ax.autoscale_view()
                plt.draw()
                plt.pause(0.01)
    except KeyboardInterrupt:
        print("\n🔴 Stopped by user.")
    finally:
        ser.close()
        plt.ioff()
        plt.show()
        print("🔌 Serial connection closed.")

```

Coding (Arduino):

```arduino
#include <Wire.h>
#include <MPU6050.h>

MPU6050 imu;

void setup() {
  Serial.begin(9600);
  Wire.begin();
  imu.initialize();

  if (imu.testConnection())
    Serial.println("MPU6050 connected!");
  else {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }
  delay(1000);
}

void loop() {
  int16_t ax, ay, az, gx, gy, gz;
  imu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  // Send data in CSV format: ax,ay,az,gx,gy,gz
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(",");
  Serial.print(az); Serial.print(",");
  Serial.print(gx); Serial.print(",");
  Serial.print(gy); Serial.print(",");
  Serial.println(gz);

  delay(50); // 20 samples/sec
}
```

5.0 DATA COLLECTION



6.0 DATA ANALYSIS

The data collected from the MPU6050 sensor represents the real-time acceleration values along the X- and Y-axes, captured and plotted through the Python program. The Arduino continuously transmitted these acceleration readings in a comma-separated format, which were then visualized using Matplotlib to show the motion pattern.

As illustrated in the plotted graph, the data points appear clustered within a narrow range, indicating that the sensor experienced minimal movement during data collection. The X-axis acceleration values fluctuate approximately between -15000 and 0, while the Y-axis values vary between -12000 and 0. This limited variation suggests that the MPU6050 was mostly static or moved only slightly within a small region.

The smooth trace without large deviations also indicates that the serial communication between Arduino and Python was stable and continuous, allowing real-time updates without noticeable delay. Any noise or minor drift in the readings can be attributed to the sensor's inherent sensitivity and small vibrations during measurement.

7.0 RESULTS

The experiment successfully demonstrated real-time motion tracking using the MPU6050 sensor, Arduino Uno, and Python visualization. The Arduino was able to continuously acquire acceleration data from the sensor and transmit it to the computer via serial communication, where it was plotted live using Matplotlib.

Figure 1 shows the real-time motion plot of the MPU6050 sensor based on the captured X- and Y-axis acceleration values. The plotted points form a small clustered pattern, indicating that the sensor remained mostly stationary with slight movements detected.

Parameter	Observed Range
X-axis Acceleration	−15000 to 0
Y-axis Acceleration	−12000 to 0
Sampling Rate	~20 samples per second

The graph updates dynamically as the sensor moves, confirming that the system correctly visualizes real-time changes in acceleration. Minor fluctuations observed in the plotted path suggest natural sensor noise or micro vibrations. Overall, the integrated system functioned reliably, with consistent serial communication and responsive graphical updates.

8.0 DISCUSSION

The experiment demonstrated how serial communication can be used to interface sensors with a computer for real-time visualization. The MPU6050 proved to be a reliable motion sensor, though sensitive to external vibrations and orientation changes. Python provided a flexible platform for live plotting, but the detection algorithm could be improved by adding numerical pattern recognition instead of visual confirmation. This experiment forms the basis for motion-based verification, which is integrated with RFID in Part 2 for a smart access system.

9.0 CONCLUSION

The objective of capturing and visualizing real-time motion data from the MPU6050 sensor was achieved. The sensor successfully tracked hand movement, and circular gestures were detected using Python visualization. This experiment strengthened understanding of I2C communication and serial data handling between Arduino and Python.

10.0 RECOMMENDATIONS

1. Implement moving average filters to reduce noise.
2. Include gyroscope data for more accurate motion pattern detection.
3. Optimize Python script to auto-detect circular motion algorithmically.
4. Use higher baud rate for smoother live plotting.
5. Calibrate the sensor for consistent readings.

Part 2: RFID + Servo Access System (Task 2: Integrated Smart Access System)

1.0 INTRODUCTION

Access control systems traditionally rely on single-factor verification, such as RFID tags or passwords. This experiment introduces an enhanced method by combining RFID identification with motion gesture verification, adding a behavioural factor for improved security.

The Arduino Uno serves as the communication hub, handling servo actuation and LED signalling, while Python acts as the control layer that validates RFID UIDs and interprets motion patterns. When both conditions are satisfied, the servo unlocks, and the green LED indicates access granted.

2.0 MATERIALS AND EQUIPMENT

No	Component
1	Arduino Uno
2	MPU6050 Sensor
3	RFID reader (MFRC522) + RFID tags/cards
4	Jumper Wires
5	Breadboard
6	Servo motor
7	Red & Green LEDs + resistors

3.0 EXPERIMENTAL SETUP

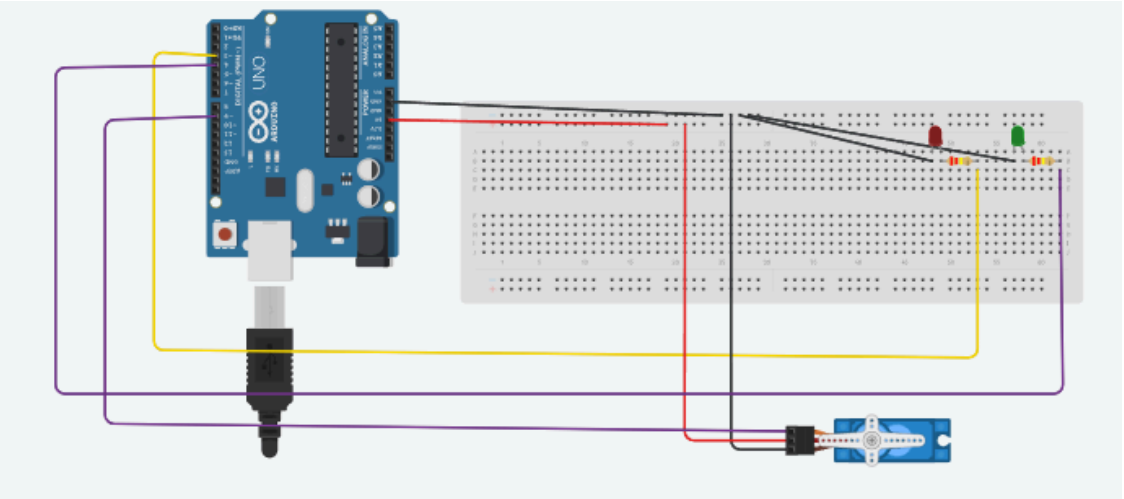


Fig 2: Servo Motor connection

Device	Pin	Arduino Uno Pin
RFID	USB Port (Laptop)	-
Servo Signal	PWM	D9
Green LED	Output	D4
Red LED	Output	D3
MPU6050 SDA	I2C	A4
MPU6050 SCL	I2C	A5
Power (5V, GND)	Shared	5V & GND

4.0 METHODOLOGY

This task integrates an RFID reader, MPU6050 motion sensor, Arduino Uno, and a Python control interface to verify user access and detect motion-based activation. The system is designed to only activate the servo motor after a valid RFID card is scanned and a circular motion is detected using the IMU sensor.

The RFID reader was connected to the computer as a USB keyboard emulator, where card IDs were detected through Python.

System Operation

1. The Python script continuously monitors keyboard input for RFID numbers typed by the reader.

2. When the user presses “Enter,” the Python program sends the detected RFID ID to the Arduino via the serial connection.

3. The Arduino program compares the received RFID ID with a predefined valid ID (0013004466).

    - If the ID does not match, the red LED turns on briefly to indicate invalid access.

    - If the ID is correct, the system requests a circular motion detected by the MPU6050 sensor.

4. Once the IMU measures a rotation magnitude above the threshold (3000), the Arduino activates the servo motor (rotates 90°) and turns on the green LED to indicate successful motion verification.

5. After a short delay, the servo returns to its original position, and the system resets to standby mode.

The Python script also provides feedback to the user in real-time through the console, displaying messages such as “Access Granted,” “Invalid Card,” and “Circular Motion Detected.”

Coding (Python):

```python
import serial
import time
import keyboard

# Change this to your Arduino port
arduino = serial.Serial('COM5', 9600, timeout=1)
time.sleep(2)  # Wait for Arduino to reset

print("RFID Reader active — Scan your card!\n")

try:
    rfid = ""
    while True:
        event = keyboard.read_event()
        if event.event_type == keyboard.KEY_DOWN:
            if event.name == 'enter':
                print(f"➡️  Sending RFID: {rfid}")
                arduino.write((rfid + '\n').encode())
                time.sleep(0.5)

                # Read Arduino response
                response = arduino.read_all().decode(errors='ignore').strip()
                if "Access Granted" in response:
                    print("✅ Access Granted!")
                elif "Invalid Card" in response or "Unknown" in response:
                    print("❌ Invalid card detected!")
                elif response:
                    print("Arduino:", response)
                rfid = ""
            else:
                rfid += event.name

except KeyboardInterrupt:
    print("\nProgram stopped.")
finally:
    arduino.close()
    print("🔌 Serial connection closed.")
```

Coding (Arduino):

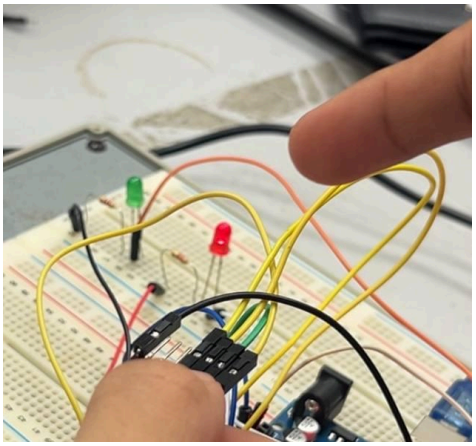
```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 imu;
Servo gateServo;

// Pin definitions
const int RED_LED = 3;
const int GREEN_LED = 4;
const int SERVO_PIN = 9;

// Variables
bool rfidMatched = false;
bool motionDetected = false;
unsigned long motionStart = 0;

void setup() {
  Serial.begin(9600);
  Wire.begin();

  // Initialize MPU6050
  imu.initialize();
  if (imu.testConnection()) {
    Serial.println("✅ MPU6050 connected!");
  } else {
    Serial.println("❌ MPU6050 connection failed!");
    while (1);
  }

  // Initialize pins
  pinMode(RED_LED, OUTPUT);
```

```
  pinMode(GREEN_LED, OUTPUT);
  gateServo.attach(SERVO_PIN);
  gateServo.write(0); // initial position

  // Initial state — all OFF
  digitalWrite(RED_LED, LOW);
  digitalWrite(GREEN_LED, LOW);

  Serial.println("System ready... Waiting for RFID input");
}

void loop() {
  // --------------- RFID Section ----------------
  if (Serial.available()) {
    String rfid = Serial.readStringUntil('\n');
    rfid.trim();

    Serial.print("Received RFID: ");
    Serial.println(rfid);

    if (rfid == "0013004466") {
      Serial.println("✅ Access Granted!");
      rfidMatched = true;
      Serial.println("Now perform a circular motion...");
    } else {
      Serial.println("❌ Invalid Card");
      digitalWrite(RED_LED, HIGH);
      digitalWrite(GREEN_LED, LOW);
      delay(2000);
      digitalWrite(RED_LED, LOW);
      rfidMatched = false;
    }
```

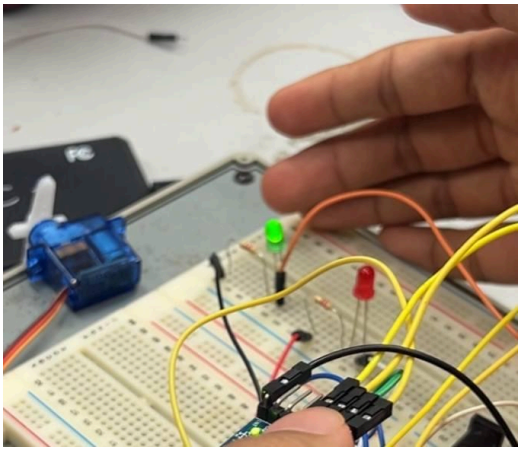
```
  }

  // --------------- IMU Section ---------------
  if (rfidMatched) {
    int16_t gx, gy, gz;
    imu.getRotation(&gx, &gy, &gz);
    float magnitude = sqrt((float)gx * gx + gy * gy + gz * gz);

    Serial.print("Rotation magnitude: ");
    Serial.println(magnitude);

    if (magnitude > 3000) { // Adjust threshold if needed
      if (!motionDetected) {
        motionDetected = true;
        Serial.println("⚙️ Circular motion detected! Activating servo and LED...");

        digitalWrite(GREEN_LED, HIGH);
        digitalWrite(RED_LED, LOW);

        gateServo.write(90);
        delay(1000);
        gateServo.write(0);

        delay(3000);
        digitalWrite(GREEN_LED, LOW);
        motionDetected = false;
        rfidMatched = false;
        Serial.println("🔴 Reset to standby");
      }
    }
  }
  delay(100);
}
```

5.0 DATA COLLECTION

```
➡️ Sending RFID: 0013004466
✅ Access Granted!
➡️ Sending RFID: 0013004466
✅ Access Granted!
➡️ Sending RFID: 0008089233
❌ Invalid card detected!
➡️ Sending RFID: 001004466
❌ Invalid card detected!
➡️ Sending RFID: 0013004466
✅ Access Granted!
➡️ Sending RFID: 0013004466
✅ Access Granted!
```



Red led for invalid card



green led for access granted

6.0 DATA ANALYSIS

The system's behavior was analyzed based on serial communication feedback and observed LED/servo responses.

- When a valid RFID (0013004466) was scanned, the Arduino printed confirmation of access and initiated IMU monitoring.
- The IMU readings (gx, gy, gz) were used to calculate the rotation magnitude, which remained below 1000 when static and rose above 3000 during circular motion.
- When the threshold was exceeded, the servo moved to 90°, and the green LED illuminated, confirming correct motion detection.

- Scanning an invalid card triggered the red LED without servo movement, validating the system's correct conditional operation.

This data confirms that the integration between Python and Arduino functioned correctly — RFID verification occurred through Python input, while motion verification and actuator control were handled by Arduino.

7.0 RESULTS

The system successfully demonstrated two-level verification using RFID authentication and IMU-based motion detection. The interaction between Python and Arduino through serial communication was reliable and responsive.

Test Condition	LED Indicator	Servo Movement	System Response
Invalid card scanned	Red ON	No	"❌ Invalid Card" printed
Valid card (no motion)	None	No	Waiting for motion
Valid card + circular motion	Green ON	90° rotation	"✅ Access Granted" + motion detected
System reset	All OFF	Servo at 0°	Ready for next input

Overall, the system achieved its objective — to only activate the servo and green LED after a correct RFID scan and a valid motion gesture. The responses from both hardware (LEDs, servo) and software (Python console messages) confirmed full integration and accurate functionality.

8.0 DISCUSSION

The experiment demonstrated how serial communication can be used to interface sensors with a computer for real-time visualization. The MPU6050 proved to be a reliable motion sensor, though sensitive to external vibrations and orientation changes. Python provided a flexible platform for live plotting, but the detection algorithm could be improved by adding numerical pattern recognition instead of visual confirmation. This experiment forms the basis

for motion-based verification, which is integrated with RFID in Part 2 for a smart access system.

9.0 CONCLUSION

The objective of capturing and visualizing real-time motion data from the MPU6050 sensor was achieved. The sensor successfully tracked hand movement, and circular gestures were detected using Python visualization. This experiment strengthened understanding of I2C communication and serial data handling between Arduino and Python.

10.0 RECOMMENDATIONS

1. Implement moving average filters to reduce noise.
2. Include gyroscope data for more accurate motion pattern detection.
3. Optimize Python script to auto-detect circular motion algorithmically.
4. Use higher baud rate for smoother live plotting.
5. Calibrate the sensor for consistent readings.

11.0 REFERENCES

LME Editorial Staff. (2025a, April 7). *What is RFID? How It Works? Interface RC522 RFID Module with Arduino*. Last Minute Engineers. https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/

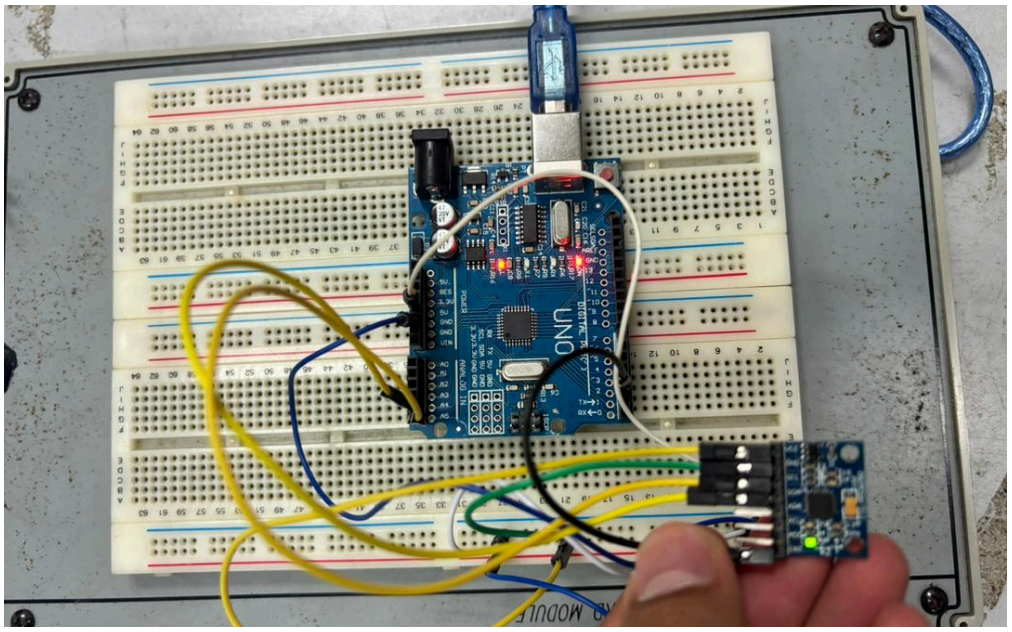
Santos, R., & Santos, R. (2023, April 23). *MFRC522 RFID Reader with Arduino Tutorial | Random Nerd Tutorials*. Random Nerd Tutorials. https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

Santos, S., & Santos, S. (2019, April 2). *Arduino Time Attendance System with RFID | Random Nerd Tutorials*. Random Nerd Tutorials. https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/

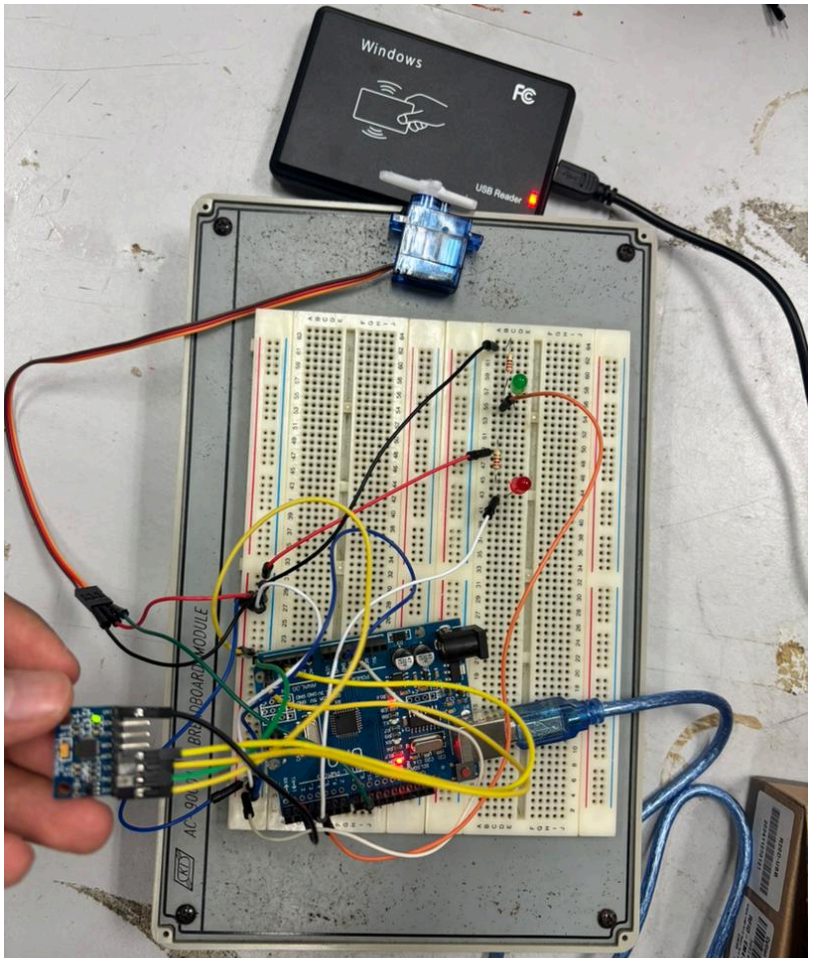
Instructables. (2017a, September 22). *Arduino + MFRC522 RFID READER*. Instructables. https://www.instructables.com/Arduino-MFRC522-RFID-READER/

12.0 APPENDICES

Task 1



Task 2



3.0 ACKNOWLEDGEMENT

We would like to express our gratitude to my lab instructor, Dr Zulkifli bin Zainal Abidin, for his guidance during the experiment. Special thanks to my groupmates and peers for their collaboration and help during circuit setup and troubleshooting.

14.0 STUDENTS DECLARATION

CERTIFICATE OF ORIGINALITY AND AUTHENTICITY

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or a person.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We, therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature: Read /
Name: Adam Nabil Haniff bin Ruzaimi Understand /
Matric Number: 2313203 Agree /
Contribution: Introduction, equipments and results

Signature: Read /
Name: Muhammad Naufal Hakimi bin Irwan Affandi Understand /
Matric Number: 2313041 Agree /
Contribution: Procedure, results, discussion

Signature: Read /
Name: Muhamad Iqbal bin Md Isa Understand /
Matric Number: 2313247 Agree /
Contribution: : Results, discussion, conclusion